

Deciphering the DSA submission bottleneck

Diagnostic target
missing cycles under full-WQ pressure

Signal	Scale	Meaning
Unloaded submit	20 cycles	posted/admitted write; credits available
Single request	sub-μs visible completion	N00P 379 ns; 64B 575 ns
Full-loop async	8–10 Mops/s	closed-loop cost, not isolated submit

cheap submit ≠ cheap closed loop

Sources: submit-only study 2026-05-24; Understanding the Host Network; direct latency measurement.

Host path view: what to measure

Where credits live

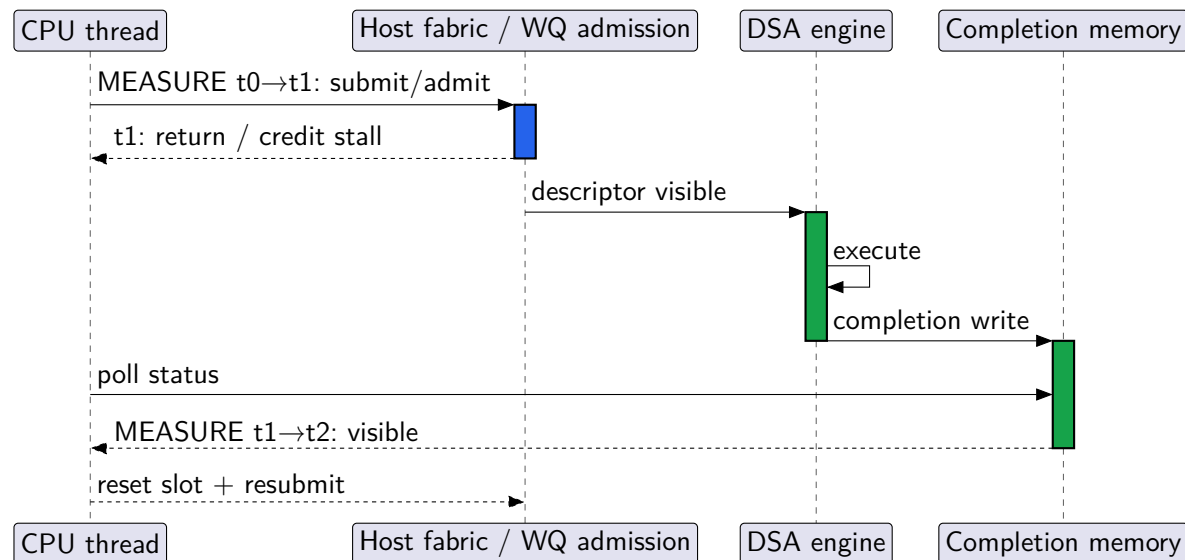
domain	processor / memory / peripheral path
credits	outstanding request budget
unloaded latency	cheap path when credits exist
wait	upstream work gets slowed

DSA implication

movdir64b returns	the WQ accepted the write
not proved	DSA execution
not proved	completion writeback
not proved	CPU can see completion
full-loop cost	first place that runs out of credits

Design rule: vary one pressure source → find first bend

The DSA loop: where timestamps attach



t0 → t1 submit / waiting

t1 → t2 work done + visible

lifeline fill = measured region

■ t0→t1 submit interval

■ t1→t2 done-and-visible interval

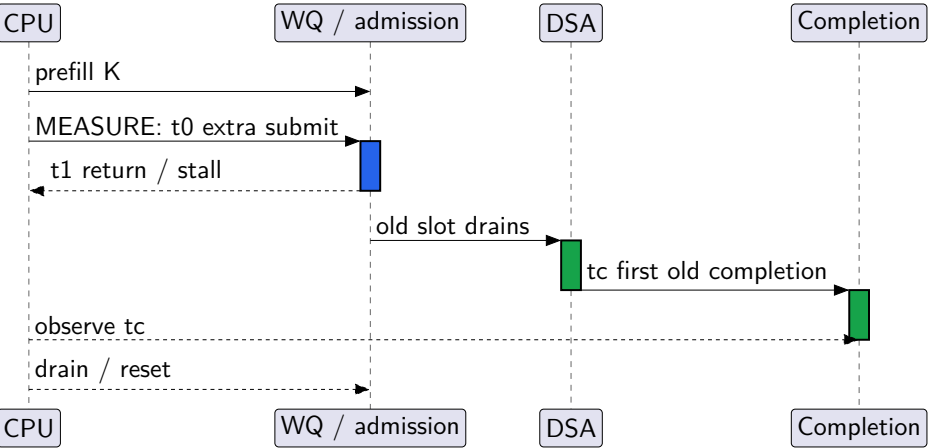
DSA traffic classes

DSA action	Host path	What it tests
movdir64b submit	CPU → peripheral posted write	t0→t1 submit slowdown
DSA source read	Peripheral → memory read	payload-read pressure
DSA destination write	Peripheral → memory write	write-buffer / fabric pressure
DSA completion write	Peripheral → memory write, cacheline-scale	completion becomes visible
CPU poll/reset	CPU memory read/write	cacheline ownership; slot reuse

Exp. 3: add classes **Exp. 5:** collide classes

Experiment 1: submit gets slow when WQ is full?

Setup



Readout

submit latency vs. occupancy

lifeline fill = measured region

■ extra-submit timer

■ first old-slot completion wait

flat	submit credits remain
knee near 128	WQ/device credits
early knee	store / posting / IIO / portal
full-WQ tail	wait for device progress

x	K = 0,32,64,96,112,120,124,127,128
y	extra-submit cycles
hold fixed	op, NUMA, WQ, core

Fill WQ more → see when submit gets slow.

Why run it

why	fill the WQ, then time one extra submit
learn	when the extra submit gets slow
record	K, submit cycles, first old done time

If submit stays fast until the queue is nearly full, the 20-cycle number is still real. If it gets slow early, the host/WQ path is already running out of credits.

Experiment 1: why run it

Simple Rust sketch

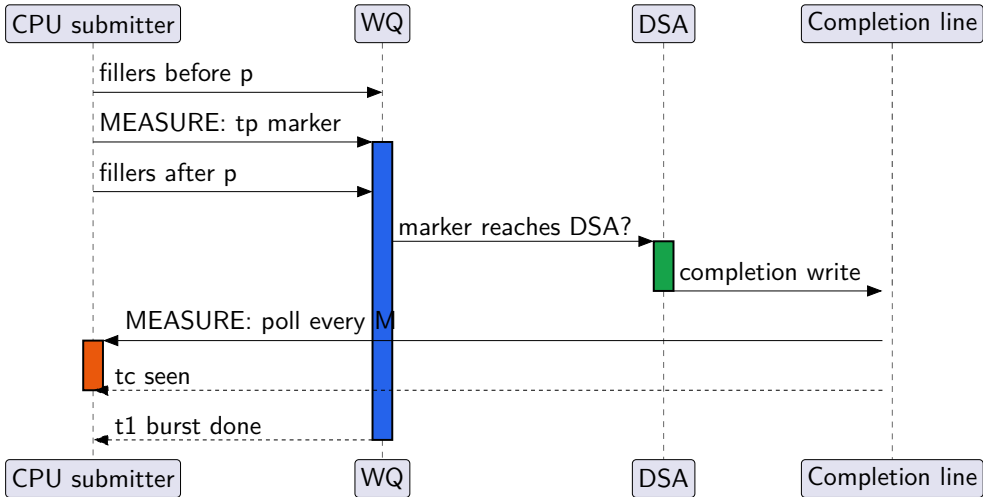
rust

```
for &k in &[0, 32, 64, 96, 112, 120, 124, 127, 128] {  
    bench.reset_slots();  
    bench.prefill(k, Op::Noop);  
  
    let t0 = ticks();  
    bench.submit_one(Op::Noop);  
    let t1 = ticks();  
  
    let first_done = bench.poll_first_old();  
    bench.drain_all();  
  
    record("credit-knee", k, t1 - t0, first_done);  
}
```

These are simple calls; the real runner still sets descriptors, fences, and checks completions.

Experiment 2: marker completion during submit

Setup



Readout

$$\text{overlap} = t_c < t_1$$

lifeline fill = measured region

■ remaining submit window

■ marker device/completion path

■ poll every M

$t_c < t_1$

device overlaps submit

only tight M

poll-sensitive

no $t_c < t_1$

late completion or long latency

same core; one TSC stream

p	1, N/2, N
M	1, 4, 16, 64, never
fillers	no-completion NOOP first

Experiment 2: why run it

Simple Rust sketch

rust

Why run it

why	a burst can hide whether DSA work overlaps CPU submission
learn	whether marker completion appears before the submit loop ends
control	poll every M submits to test polling cost

A marker that completes before t1 means DSA work overlaps the submit tail. If the answer changes only with tight polling, polling is changing the result.

```
for p in [1, n / 2, n] {
    for m in [1, 4, 16, 64, usize::MAX] {
        bench.reset_slots();
        let mut tc = None;

        for i in 0..n {
            let op = if i == p { Marker } else { NoComp };
            bench.submit_one(op);

            if i % m == 0 {
                tc = tc.or_else(|| bench.poll_s());
            }
        }

        let t1 = ticks();
        tc = tc.or_else(|| bench.wait_s());
        record("marker", p, m, tc.unwrap() < t1);
    }
}
```


Polling can change the result

What can go wrong

1. CPU keeps reading status = 0

This is not free: it is a tight load loop.

2. DSA writes done

The completion line changes owner / state.

3. CPU resets the same line

Reusing the slot may add cacheline traffic.

Why this control exists

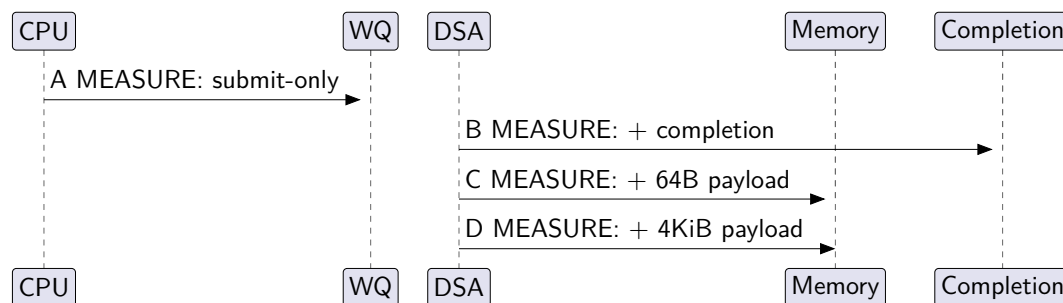
Are we measuring DSA, or are we measuring our polling loop?

poll less	does the result move?
poll tight	best possible notice time
delay reset	separate reset cost
pad slots	test cacheline sharing

if polling changes throughput, fix polling first

Experiment 3: add one traffic type at a time

Traffic ladder



A	submit-only / no completion
B	completion-bearing NOOP
C	64B memmove
D	4KiB memmove

Readout

same curves; one class added

lifeline fill = measured region

■ submit-only path

■ completion path

■ 64B payload path

■ 4KiB payload path

A bends	submit alone
B adds	completion write / seen by CPU
C/D add	payload DMA domain
D unique	fabric / bandwidth / locality

Experiment 3: why run it

Simple Rust sketch

rust

Why run it

why	change one thing per run
learn	which added work makes it slow
compare	same WQ, core, window, NUMA node

This is the simple ladder: submit only, then completion writes, then payload reads/writes. The first new slowdown says what to check next.

```
let classes = [
    Op::SubmitOnly,
    Op::Noop,
    Op::Memcpy { bytes: 64 },
    Op::Memcpy { bytes: 4096 },
];

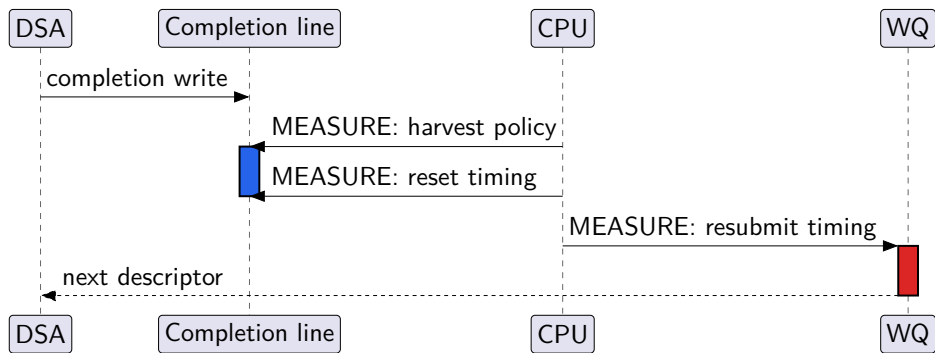
for op in classes {
    for window in [1, 8, 32, 64, 96, 128] {
        bench.reset_slots();

        let curve = bench.run_window(op, window, reps);
        record_curve("traffic", op, window, curve);
    }
}
```

The runner changes only the operation class; all pacing and measurement code stays shared.

Experiment 4: completion handling loop

Reuse loop



layout	packed / padded
poll	scan-all / round-robin
reset	none / delayed / immediate
submit	per-completion / batch harvest

Readout

throughput sensitivity to reuse policy

lifecycle fill = measured region
completion-line harvest
completion-line reset
WQ resubmit

padding helps	ownership / false sharing
scan hurts	poll load stream
batch helps	burstiness / cache churn
no movement	lower-level path

Experiment 4: why run it

Simple Rust sketch

rust

Why run it

why	CPU completion handling can be the slow part
learn	which CPU policy changes throughput
record	ops/sec, polls/done, reset-to-submit cycles

If padding, scan order, or batch harvesting changes the curve, fix the CPU completion path before changing hardware submission.

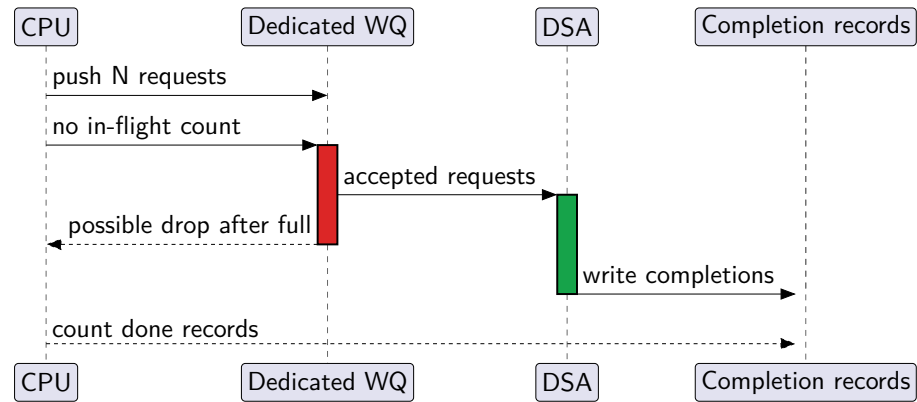
```
for policy in [
    PackedScan,
    PaddedRoundRobin,
    PollOnly,
    DelayedReset,
    BatchHarvest,
] {
    bench.configure(policy);
    bench.fill_window(128, Op::Noop);

    while bench.elapsed() < run_for {
        let done = bench.harvest(policy);
        bench.reset_done(done, policy);
        bench.resubmit(done, Op::Noop);
    }

    record("reuse", policy, bench.ops_per_sec(), bench.polls_per_done());
}
```

Experiment 5: push without counting

Setup



Readout

completed count vs. pushed count

lifeline fill = measured region
■ push past WQ depth
■ accepted requests complete

same count	no loss seen in this run
missing ids	some pushes were dropped
loss after depth	WQ full behavior
SWERROR set	hardware noticed bad submit

N	depth/2, depth, 2×depth, 4×depth
op	N00P first; optional 64B memmove
rule	do not poll while pushing

This intentionally stops counting in-flight work to test whether requests can disappear.

Why run it

why	movdir64b does not tell us “WQ accepted it”
learn	whether pushing without counting loses requests
record	pushed, completed, missing ids, SWERROR

If pushed count is larger than completed count, then we need in-flight tracking before trusting any throughput number from a dedicated WQ.

Experiment 5: why run it

Simple Rust sketch

rust

```
for n in [depth / 2, depth, 2 * depth, 4 * depth] {  
    bench.reset_unique_slots(n);  
  
    for id in 0..n {  
        bench.submit_no_wait(id, Op::Noop);  
    }  
  
    let done = bench.wait_done_until(timeout);  
    let missing = bench.missing_ids(0..n);  
    let swerr = bench.read_software_error();  
  
    record("no-count-push", n, done.len(), missing, swerr);  
}
```

Use one completion slot per pushed request; otherwise slot reuse can hide loss.

Attribution rule

First bend	Credit domain	Next probe
one-submit vs K	submit / WQ accept	knee depth vs WQ size
early marker	hardware overlaps submit	drain / reuse loop
late marker	posting / completion delay	IIO / WQ credits
reuse variants	poll / reset / cacheline	reuse redesign
no-count push loses	missing completions	restore in-flight tracking

Find first bend → name the credit domain.

Experiment 1 result: admission bends before WQ size

first plateau

$K \approx 116$

extra submit

224–226 ticks

nominal WQ

128

cheap regime

$K \leq 114$
24–32 TSC ticks

transition

$K = 115$
NOOP 86; 64B 194 ticks

backpressured plateau

$K \geq 116$
101 ns extra-submit

Evidence rows

op	K	submitted	extra TSC	status
NOOP	114	115	32	ok
NOOP	115	116	86	ok
NOOP	116	117	226	ok
64B	114	115	24	ok
64B	115	116	194	ok
64B	116	117	224	ok

Attribution

one-submit vs K bends early

64B follows NOOP → first bend is admission / credits,
not payload DMA.

completed == submitted; missing = 0; errors = 0

Next: marker overlap asks whether completions are visible during the submit tail.